Mutation Testing for SIMPLE Typechecker

Ojas Hegde, Priyansh Agrahari

November 2025

Contents

1 Introduction **2**

2 Architecture **2**

3 Mutation Operators **3**
3.1 AOR (Arithmetic Operator Replacement) 4
3.2 LCR (Literal Constant Replacement) 5
3.3 RTR (Return Type Replacement) 6

4 Results **7**
4.1 Arithmetic Operator Replacement 7
4.2 Literal Constant Replacement 7
4.3 Return Type Replacement 7

5 Conclusion **8**

6 LLM Usage Declaration **8**

1 Introduction

This project implements a Mutation Testing Framework designed to validate the robustness of a Type Checker for a custom programming language named "SIMPLE". Unlike traditional unit testing, which tests the program's output, this system tests the test suite (in this case, the Type Checker logic) by deliberately generating faulty program ASTs ("mutants").
The system parses a source file, generates semantically or syntactically altered versions of the Abstract Syntax Tree (AST), and verifies whether the Type Checker correctly accepts valid mutants and rejects invalid ones.

2 Architecture

The workflow follows a standard "Generate and Test" pipeline managed by the `MutationTesting` class.

1. Parsing & Sanity Check:
   (a) The system uses JFlex (Lexer) and CUP (Parser) to convert the source `.simple` file into a `ProgramNode` (AST).
   (b) The `TypeChecker` is run on the original AST. If the original program is already ill-typed, the process halts to prevent testing an already broken program.
2. Mutant Generation:
   (a) The `MutationTraverser` iterates through the AST.
   (b) It delegates logic to specific `Mutator` implementations (AOR, LCR, RTR).
   (c) Before applying a mutation, the system uses `AstNode.deepCopy()` to clone the specific node or sub-tree. This ensures that mutations are isolated and do not corrupt the original AST or subsequent mutants.
3. Execution & Analysis:
   (a) A fresh instance of `TypeChecker` is instantiated for every single mutant to ensure a clean Symbol Table.
   (b) **Killed Mutant:** The `TypeChecker` throws an exception. This is the desired outcome for mutants that introduce type errors.
   (c) **Survived Mutant:** The `TypeChecker` accepts the program. This occurs if the mutant is syntactically valid (e.g., changing `x + 5` to `x + 0`).

3 Mutation Operators

The project is architected around a modular `Mutator` interface, designed to decouple the specific mutation logic from the AST traversal engine. This extensible design allows for the integration of different mutation strategies without modifying the core testing framework.

The `MutationTraverser` class is responsible for orchestrating the mutation process and enforcing **mutation production coverage**. As it recursively walks the Abstract Syntax Tree (AST), it identifies every node that matches a specific operator's target criteria (e.g., every instance of `BinaryExpr` for AOR). For each identified location, the system generates a comprehensive list of all possible mutants defined by that operator. This exhaustive approach ensures that the Type Checker is stress-tested against every applicable occurrence of a fault pattern.

We'll use the below AST to demonstrate the different mutations:

Listing 1: Original AST

```
ProgramNode:
Definitions:
  Func: f Params: VarDecl: INTEGER x VarDecl: BOOLEAN y
  Body:
    BlockStmt:
      VarDecl: INTEGER z
    Statements
      If (Id: y )
      Then:
        z := Id: x ADD IntLiteral: 10
      Else:
        z := Id: x SUB IntLiteral: 5
      ReturnStmt: Id: z
Globals:
  VarDecl: INTEGER x
  VarDecl: INTEGER y
Main:
  x := IntLiteral: 10
  y := FuncCall: f Args: (Id: x BoolLiteral: true )
  x := FuncCall: f Args: (Id: y BoolLiteral: false )
```

3.1 AOR (Arithmetic Operator Replacement)

Target: BinaryExpr nodes.

Logic: Replaces Arithmetic Operators (ADD, SUB, MUL, DIV) with Boolean Operators (AND, OR).

Objective: To ensure the Type Checker catches type errors where a Boolean is used in place of an Integer (or vice versa).

Listing 2: AOR Mutated AST (Killed)

```
ProgramNode:
Definitions:
  Func: f Params: VarDecl: INTEGER x VarDecl: BOOLEAN y
  Body:
    BlockStmt:
      VarDecl: INTEGER z
    Statements
      If (Id: y )
      Then:
        z := Id: x AND IntLiteral: 10
      Else:
        z := Id: x SUB IntLiteral: 5
      ReturnStmt: Id: z
Globals:
  VarDecl: INTEGER x
  VarDecl: INTEGER y
Main:
  x := IntLiteral: 10
  y := FuncCall: f Args: (Id: x BoolLiteral: true )
  x := FuncCall: f Args: (Id: y BoolLiteral: false )
```

3.2 LCR (Literal Constant Replacement)

Target: IntLiteral and BoolLiteral nodes.

Logic:

- Value Replacement: Changes values (e.g., 5 → 0, true → false). These usually Survive because they are valid types.
- Type Replacement: Swaps types (e.g., 5 → true). These are expected to be Killed.

Objective: To test if the Type Checker is validating the types of literals, distinguishing between valid semantic changes and invalid type changes.

Listing 3: LCR Mutated AST (Killed)

```
ProgramNode:
Definitions:
  Func: f Params: VarDecl: INTEGER x VarDecl: BOOLEAN y
  Body:
    BlockStmt:
      VarDecl: INTEGER z
    Statements
      If (Id: y )
      Then:
        z := Id: x ADD BoolLiteral:  true
      Else:
        z := Id: x SUB IntLiteral: 5
      ReturnStmt: Id: z
Globals:
  VarDecl: INTEGER x
  VarDecl: INTEGER y
Main:
  x := IntLiteral: 10
  y := FuncCall: f Args: (Id: x BoolLiteral: true )
  x := FuncCall: f Args: (Id: y BoolLiteral: false )
```

3.3 RTR (Return Type Replacement)

Target: ReturnStmt nodes.

Logic: Replaces the return expression with a fixed constant.

- Mutant 1: return 0; (Integer)
- Mutant 2: return true; (Boolean)

Objective: To verify that the Type Checker enforces function return signatures.

Listing 4: RTR Mutated AST (Killed)

```
ProgramNode:
Definitions:
  Func: f Params: VarDecl: INTEGER x VarDecl: BOOLEAN y
  Body:
    BlockStmt:
      VarDecl: INTEGER z
    Statements
      If (Id: y )
      Then:
        z := Id: x ADD IntLiteral: 10
      Else:
        z := Id: x SUB IntLiteral: 5
      ReturnStmt: BoolLiteral:  true
Globals:
  VarDecl: INTEGER x
  VarDecl: INTEGER y
Main:
  x := IntLiteral: 10
  y := FuncCall: f Args: (Id: x BoolLiteral: true )
  x := FuncCall: f Args: (Id: y BoolLiteral: false )
```

4 Results

The mutation testing system evaluated the robustness of the Type Checker using three distinct mutation operators: AOR (Arithmetic Operator Replacement), LCR (Literal Constant Replacement), and RTR (Return Type Replacement).

The objective was to verify that the Type Checker:

- Rejects (Kills) mutants that violate type rules.
- Accepts (Allows to Survive) mutants that remain well-typed.

Table 1: Summary of Performance

Mutation Operator	Total Mutants	Killed	Survived	Mutation Score
AOR	4	4	0	100%
LCR	13	8	5	61.54%
RTR	2	1	1	50%
Total	19	13	6	68.42%

4.1 Arithmetic Operator Replacement

Result: 100% Score (4/4 Killed).
Analysis: The Type Checker correctly rejected all mutants because the inserted boolean operators (AND, OR) require boolean operands, but received integers.
Conclusion: The system strictly enforces operand type compatibility for logical operations.

4.2 Literal Constant Replacement

Result: 61.54% Score (8/13 Killed, 5 Survived).
Analysis:

- **Killed (Type Replacements):** Mutants that substituted incompatible types (e.g., `int` → `boolean`) were correctly flagged as errors.
- **Survived (Value Replacements):** Mutants that changed values (e.g., `10` → `0`) survived. Since `0` is a valid `int`, the program remains well-typed despite the semantic change.

Conclusion: The results confirm the Type Checker validates structural type consistency while correctly ignoring semantic (value) correctness.

4.3 Return Type Replacement

Result: 50.00% Score (1/2 Killed, 1 Survived).
Analysis:

- The **Killed** mutant (`return true;`) violated the function's `int` signature.
- The **Survived** mutant (`return 0;`) adhered to the `int` signature, making it type-safe despite the logical error.

Conclusion: The Type Checker successfully validates return statements against the function signature.

5 Conclusion

This project successfully implemented a Mutation Testing Framework designed to rigorously validate the Type Checker of the SIMPLE programming language. By employing a "Generate and Test" architecture, the system leverages AST manipulation and deep copying to create isolated test cases that stress-test the Type Checker's logic.

The implementation of modular mutation operators Arithmetic Operator Replacement (AOR), Literal Constant Replacement (LCR), and Return Type Replacement (RTR) allowed for a multifaceted evaluation of the system. The test results offer a clear distinction between the responsibilities of a type checker versus those of a functional test suite:

- **Robustness against Type Errors:** The perfect mutation score achieved with AOR demonstrates that the Type Checker is highly effective at identifying and rejecting fundamental type mismatches, such as using boolean operators on integer operands.
- **Precision of Scope:** The surviving mutants in the LCR and RTR tests highlight the intended boundaries of static analysis. The Type Checker correctly accepts programs that are syntactically and typographically valid, even when their runtime semantics (values) are logically incorrect.

Ultimately, this project confirms that Mutation Testing is an effective methodology not only for testing software applications but also for verifying the correctness and precision of static analysis tools.

6 LLM Usage Declaration

LLMs were utilized to assist in the development of this project and the preparation of this report:

- **Code Assistance:** Debug and troubleshoot the implementation of the Mutation Testing framework.
- **Report Generation:** Articulate the system architecture and generate verbose descriptions for the results analysis.

All code and text generated were reviewed and verified by the authors to ensure accuracy.